Computer Programming 1

Lecture – Structures

(16-10-2026)

Outline

- The Structures
- Operations on Structures
- Recursive Structures
- Ordered Arrays of Structures

Structure

- A structure is an ordered collection of non-homogeneous elements
  - Elements are called members or fields
  - Each field has a specific type, name and value
- A structure allows you to define new types of aggregate objects
  - The structure can be used as a single object
  - The fields can be used individually
- Each field can itself be of a type structure (nested Struct)

Structure

- A structure is an ordered collection of non-homogeneous elements
  - Elements are called members or fields
  - Each field has a specific type, name and value
- A structure allows you to define new types of aggregate objects
  - The structure can be used as a single object
  - The fields can be used individually
- Each field can itself be of a type structure (nested Struct)

Definition of the type Struct

A new aggregated type is defined

- Syntax (definition):

Named struct

```
struct newStruct_ID {
  type1 field1;
  ...
  typeN fieldN;
  };
```

- Syntax (use):

```
newStruct_ID   varID;
```

- E.g.,

```
struct complex {          // definition of the type "complex"
  double re1;             // member/field of type double (real number)
  double re2;             // member/field of type double (real number)
};
complex c,c1;             // definition of two variables of type "complex"
```

Variant:

Non-named struct

```
struct {
  type1 field1;
  ...
  typeN fieldN;
  } varID ;
```

E.g., variant:

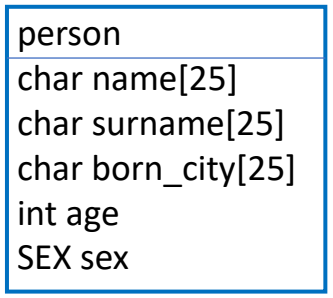
```
struct {
  double re1;
  double re2;
} c, c1;
```

Examples of Struct definitions (summing up)

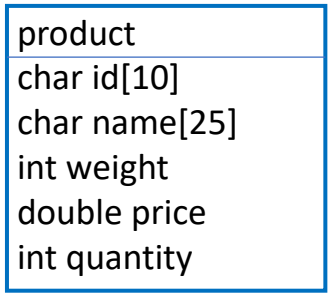
```
enum SEX { F, M, NotDeclared};
struct person {
 char name[25]
 char surname[25];
 char born_city[25];
 int age;
 SEX sex;
};

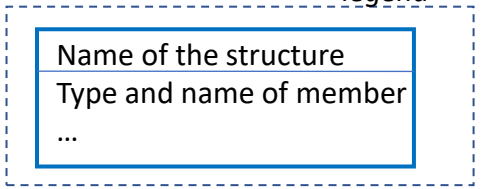
struct date {
 int day, month, year;
};

struct product {
 char id[10];
 char name[25] ;
 int weight;
 double price;
 int quantity;
};
```









Examples of Struct definitions (summing up)

```
enum SEX { F, M, NotDeclared};
struct person {
 char name[25]
 char surname[25];
 char born_city[25];
 int age;
 SEX sex;
};

struct date {
 int day, month, year;
};

struct product {
 char id[10];
 char name[25] ;
 int weight;
 double price;
 int quantity;
};
```

Struct Name

person
char name[25]
char surname[25]
char born_city[25]
int age
SEX sex

date
int day
int month
int year

product
char id[10]
char name[25]
int weight
double price
int quantity

legend

Name of the structure
Type and name of member
…

Examples of Struct definitions (summing up)

```
enum SEX { F, M, NotDeclared};
struct person {
 char name[25]
 char surname[25];
 char born_city[25];
 int age;
 SEX sex;
};

struct date {
 int day, month, year;
};

struct product {
 char id[10];
 char name[25] ;
 int weight;
 double price;
 int quantity;
};
```

Members/fields

Members/fields

Members/fields

person
char name[25]
char surname[25]
char born_city[25]
int age
SEX sex

date
int day
int month
int year

product
char id[10]
char name[25]
int weight
double price
int quantity

legend

Name of the structure
Type and name of member
…

Examples of Struct definitions (non-named variant)

```
enum SEX { F,M,NotDeclared };
struct {
 char name[25]
 char surname[25];
 char born_city[25];
 int age;
 SEX sex;
} p1;

struct {
 int day, month, year;
} date1;

struct {
 char id[10];
 char name[25] ;
 int weight;
 double price;
 int quantity;
} prd1;
```

No struct name/id

Variable definition
(more variables can be defined separated by comma «,»)

:p1
char name[25]
char surname[25]
char born_city[25]
int age
SEX sex

:date1
int day
int month
int year

:prd1
char id[10]
char name[25]
int weight
double price
int quantity

Structure (recall)

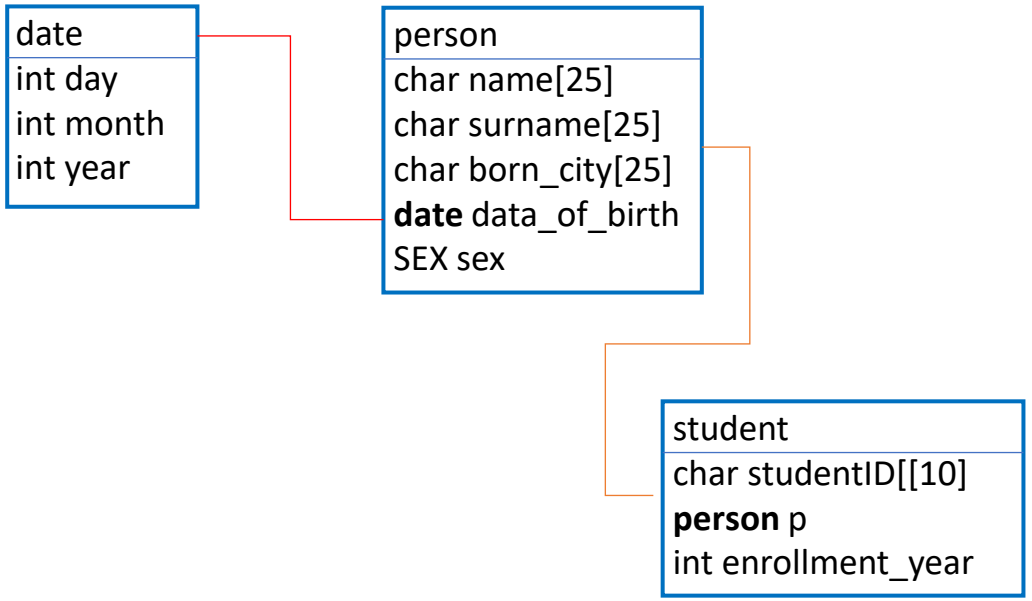
- A structure is an ordered collection of non-homogeneous elements
  - Elements are called members or fields
  - Each field has a specific type, name and value
- A structure allows you to define new types of aggregate objects
  - The structure can be used as a single object
  - The fields can be used individually
- Each field can itself be of a type structure (nested Struct)

Definition of nested Struct (1)

```
struct date {                    // simple structure
 int day, month, year;           //fields
};

enum SEX { F,M,NotDeclared };
struct person {                  // nested structure
 char name[25], surname[25];
 char born_city[25];
 date date_of_birth;
 SEX sex;
};

struct student {                 // nested structure
 person p;
 char studentID[10];
 int enrollment_year;
};
```

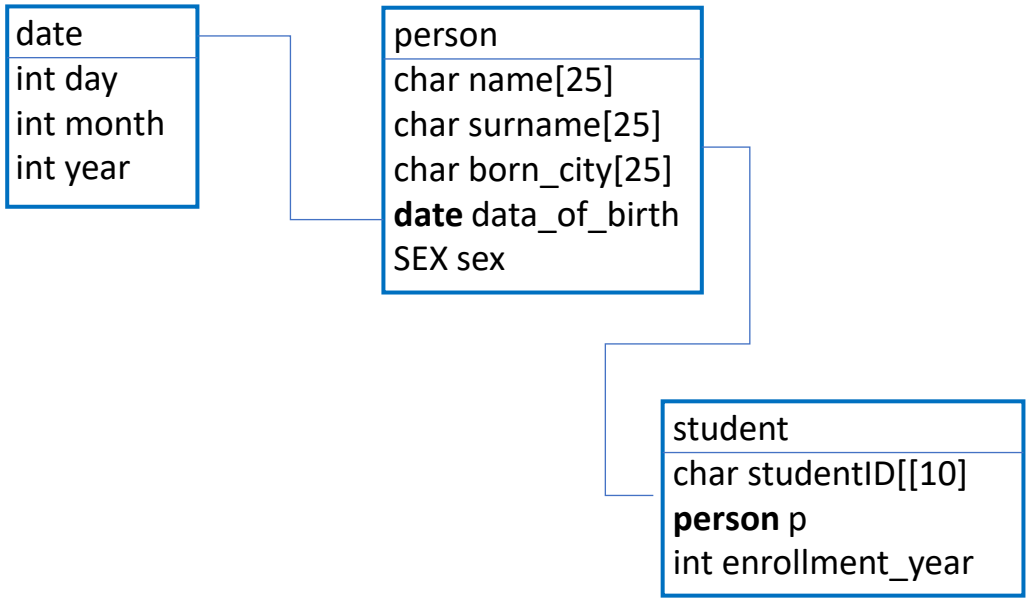


Definition of nested Struct (2)

```
enum SEX { F,M,NotDeclared };
struct person {                    // nested structure
 char name[25], surname[25];
 char born_city[25];
 //date date_of_birth;
 struct date {                     // simple structure
     int day, month, year;         //fields
 } date_of_birth;
 SEX sex;
};

struct student {                   // nested structure
 person p;
 char studentID[10];
 int enrollment_year;
};
```

Note: «struct data{ …} data_of_bird;» *data* can be omitted (i.e., Non-named struct)



Structure (recall)

- A structure is an ordered collection of non-homogeneous elements
  - Elements are called members or fields
  - Each field has a specific type, name and value
- A structure allows you to define new types of aggregate objects
  - The structure can be used as a single object
  - The fields can be used individually
- Each field can itself be of a type structure (nested Struct)

Initialization of variables member of a Struct

- A variable member of a type struct can be assigned to a default value and/or initialized

```
struct Point {
   int x = 0; // like default arguments
   int y = 1;
};

int main()
{
  Point p1; //define p1 of type Point

   //…use of p1
}
```

Example:

{ ../STRUCT/testzero.cc, struct_initialization.cc}

Initialization of variables member of a Struct

- A variable member of a type struct can be assigned to a default value and/or initialized

```
struct Point {
   int x = 0; // like default arguments
   int y = 1;
};

int main()
{
   Point p1; //define p1 of type Point

   //…use of p1
}
```

```
struct Point {
   int x, y;
};

int main()
{
   // A valid initialization.
  // member x gets value 0 and
  // y gets value 1.
 Point p1 = { 0, 1 };
}
```

Example:

{ ../STRUCT/testzero.cc, struct_initialization.cc}

Initialization of variables member of a Struct

- A variable member of a type struct can be assigned to a default value and/or initialized

```
struct Point {
   int x = 0; // like default arguments
   int y = 1;
};

int main()
{
   Point p1; //define p1 of type Point

   //…use of p1
}
```

```
struct Point {
   int x, y;
};

int main()
{
   // A valid initialization.
   // member x gets value 0 and
   // y gets value 1.
  Point p1 = { 0, 1 };
}
```

```
struct Point {
   int x, y;
};

int main()
{
  Point p1; //define p1 of type Point
  //…use of p1
}
```

E.g.,

```
int x;            //zero-initialized to 0
double d[3];      //zero-initialized to three 0.0
int* p;           // zero-initialized to null pointer value,
                  // check with: if (p != 0) {…}
string s;         //zero-initialized to default-initialized to ""
```

It is dangerous! …
usually it is better to avoid

Example:

{ ../STRUCT/testzero.cc, struct_initialization.cc}

Initialization of variable of type Struct

- A variable of type struct is initialized with ordered lists of field values
  - They must match in order and type
  - Any missing values are started at zero of the type

```
struct date {
 int day, month, year;
};


enum  SEX { F, M, NotDeclared };
struct person {
 char name[25], surname[25];
 char born_city[25];
 date date_of_birth;
 SEX sex;
};
```

Example:

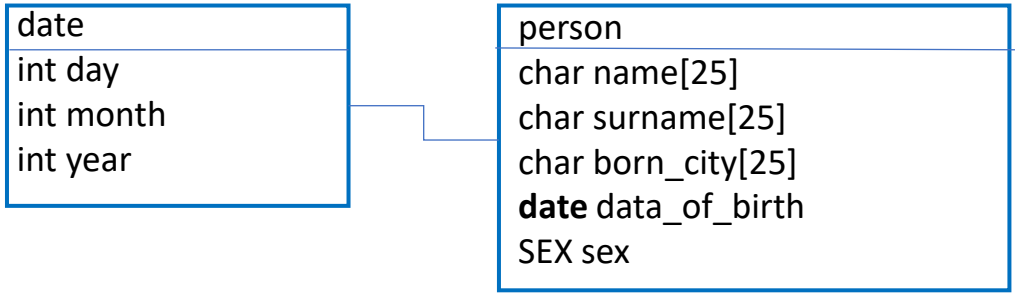
{ ../STRUCT/testzero.cc, struct_initialization.cc}

Initialization of variable of type Struct

- A variable of type struct is initialized with ordered lists of field values
  - They must match in order and type
  - Any missing values are started at zero of the type

```
struct date {
 int day, month, year;
};


enum  SEX { F, M, NotDeclared };
struct person {
 char name[25], surname[25];
 char born_city[25];
 date date_of_birth;
 SEX sex;
};
```



Example:

{ ../STRUCT/testzero.cc, struct_initialization.cc}

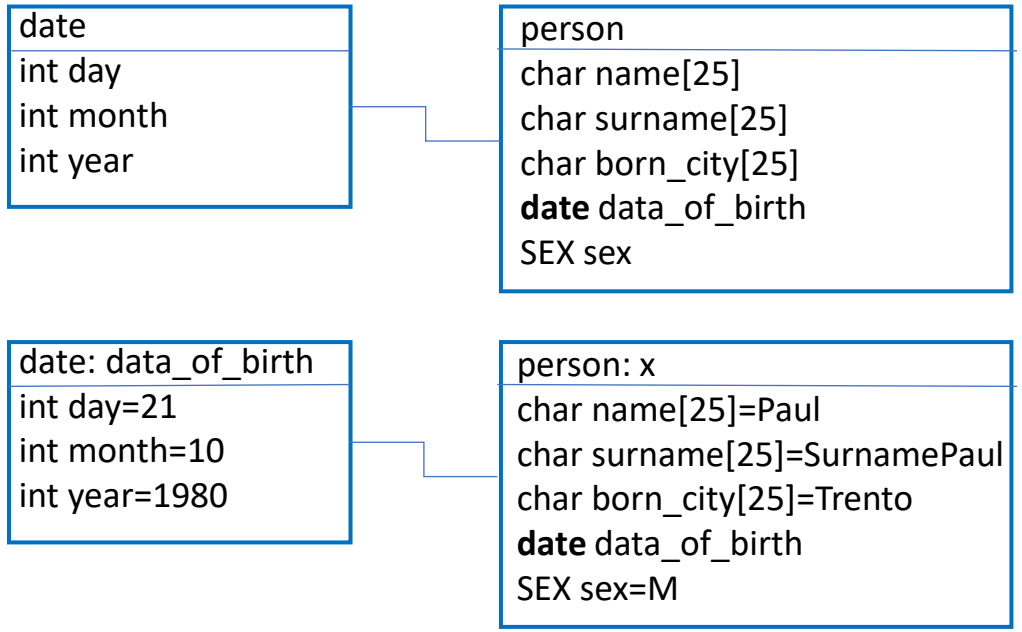
Initialization of variable of type Struct

- A variable of type struct is initialized with ordered lists of field values
  - They must match in order and type
  - Any missing values are started at zero of the type

```
struct date {
 int day, month, year;
};


enum SEX { F, M, NotDeclared };
struct person {
 char name[25], surname[25];
 char born_city[25];
 date date_of_birth;
 SEX sex;
};


person x = { “Paul”, “SurnamePaul”, “Trento”, {21, 10, 1980}, M };
```



Example:

{ ../STRUCT/testzero.cc, struct_initialization.cc}

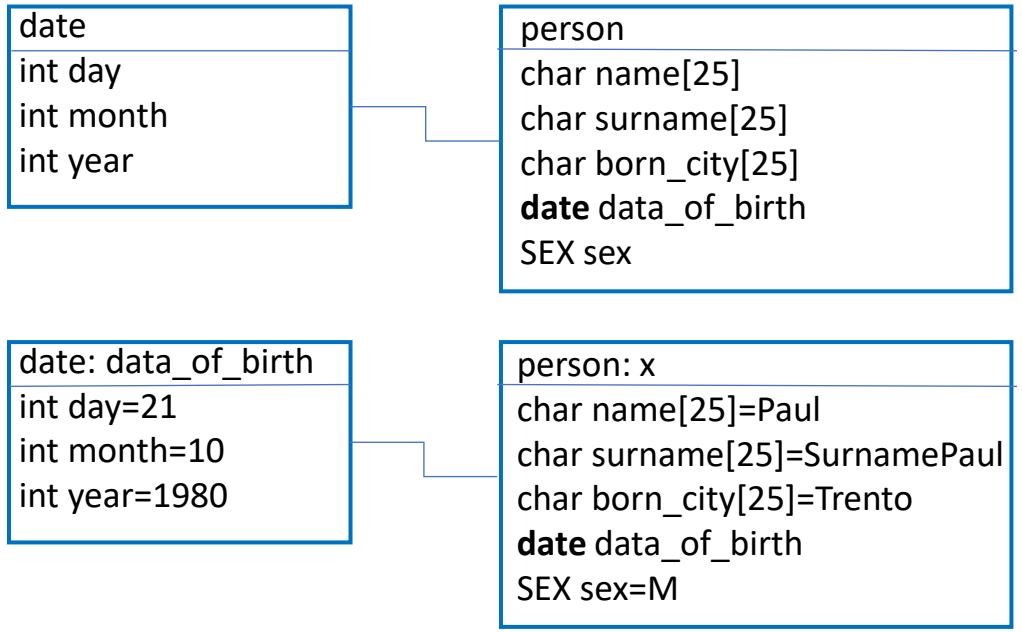
Initialization of variable of type Struct

- A variable of type struct is initialized with ordered lists of field values
  - They must match in order and type
  - Any missing values are started at zero of the type

```
struct date {
 int day, month, year;
};


enum SEX { F, M, NotDeclared };
struct person {
 char name[25], surname[25];
 char born_city[25];
 date date_of_birth;
 SEX sex;
};


person x = { “Paul”, “SurnamePaul”, “Trento”, {21, 10, 1980}, M };
```



Example:

{ ../STRUCT/testzero.cc, struct_initialization.cc}

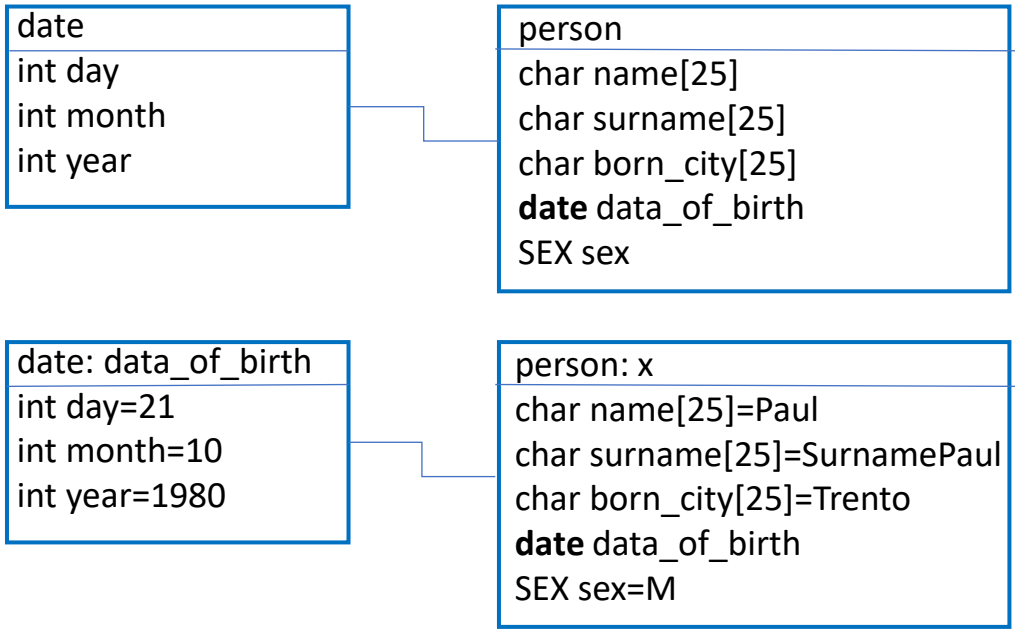
Initialization of variable of type Struct

- A variable of type struct is initialized with ordered lists of field values
  - They must match in order and type
  - Any missing values are started at zero of the type

```
struct date {
 int day, month, year;
};


enum SEX { F, M, NotDeclared };
struct person {
 char name[25], surname[25];
 char born_city[25];
 date date_of_birth;
 SEX sex;
};


person x = { “Paul”, “SurnamePaul”, “Trento”, {21, 10, 1980}, M };
```



Example:

{ ../STRUCT/testzero.cc, struct_initialization.cc}

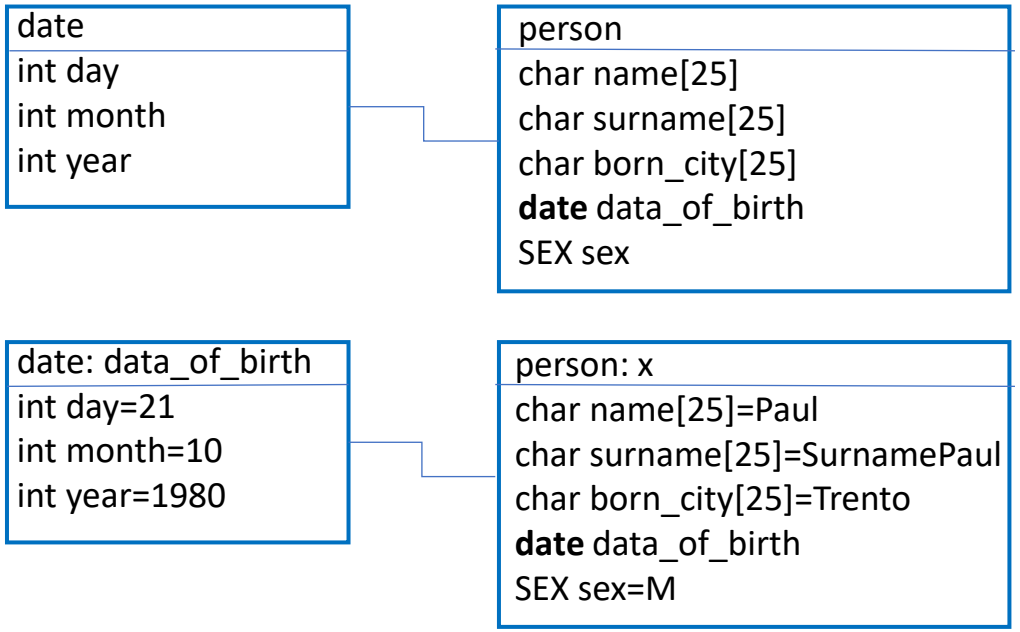
Initialization of variable of type Struct

- A variable of type struct is initialized with ordered lists of field values
  - They must match in order and type
  - Any missing values are started at zero of the type

```
struct date {
 int day, month, year;
};

enum SEX { F, M, NotDeclared };
struct person {
 char name[25], surname[25];
 char born_city[25];
 date date_of_birth;
 SEX sex;
};

person x = { “Paul”, “SurnamePaul”, “Trento”, {21, 10, 1980}, M };
```



Example:

{ ../STRUCT/testzero.cc, struct_initialization.cc}

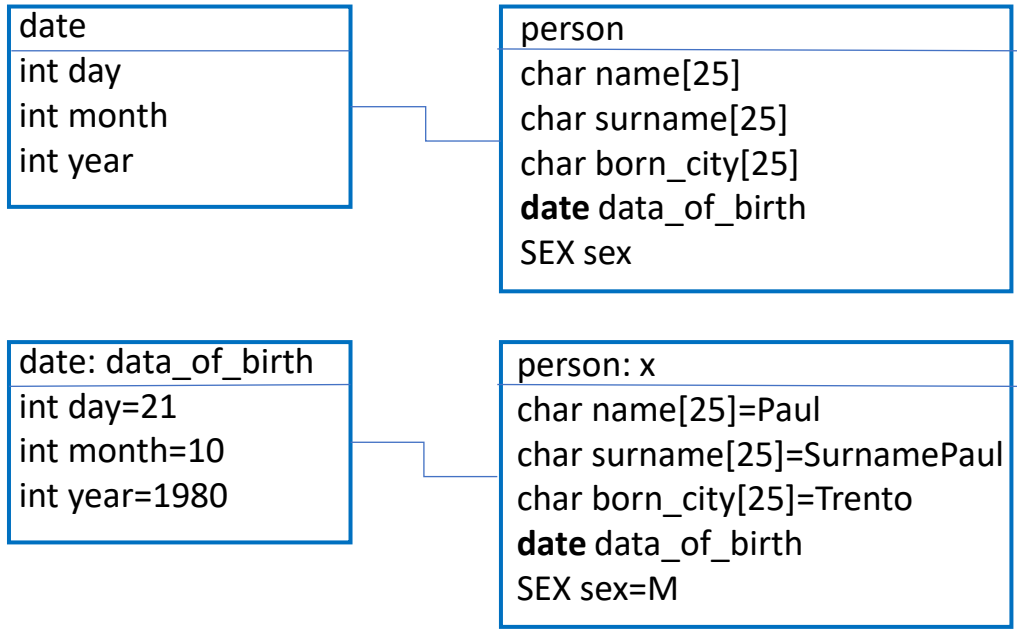
Initialization of variable of type Struct

- A variable of type struct is initialized with ordered lists of field values
  - They must match in order and type
  - Any missing values are started at zero of the type

```
struct date {
 int day, month, year;
};


enum SEX { F, M, NotDeclared };
struct person {
 char name[25], surname[25];
 char born_city[25];
 date date_of_birth;
 SEX sex;
};


person x = { “Paul”, “SurnamePaul”, “Trento”, {21, 10, 1980}, M };
person x = { "Paul", "SurnamePaul", "Trento", {21.1, 10, 1980}, M }; //compile error
```



Example:

{ ../STRUCT/testzero.cc, struct_initialization.cc}

Initialization of variable of type Struct

- A variable of type struct is initialized with ordered lists of field values
  - They must match in order and type
  - Any missing values are started at zero of the type

```
struct date {
 int day, month, year;
};


enum SEX { F, M, NotDeclared };
struct person {
 char name[25], surname[25];
 char born_city[25];
 date date_of_birth;
 SEX sex;
};

person x = { “Paul”, “SurnamePaul”, “Trento”, {21, 10, 1980}, M };
person x = { "Paul", "SurnamePaul", "Trento", {}, M };
```

day=month=year=0

date
int day
int month
int year

person
char name[25]
char surname[25]
char born_city[25]
date data_of_birth
SEX sex

date: data_of_birth
int day=21
int month=10
int year=1980

person: x
char name[25]=Paul
char surname[25]=SurnamePaul
char born_city[25]=Trento
date data_of_birth
SEX sex=M

Example:

{ ../STRUCT/testzero.cc, struct_initialization.cc}

Initialization of variable of type Struct

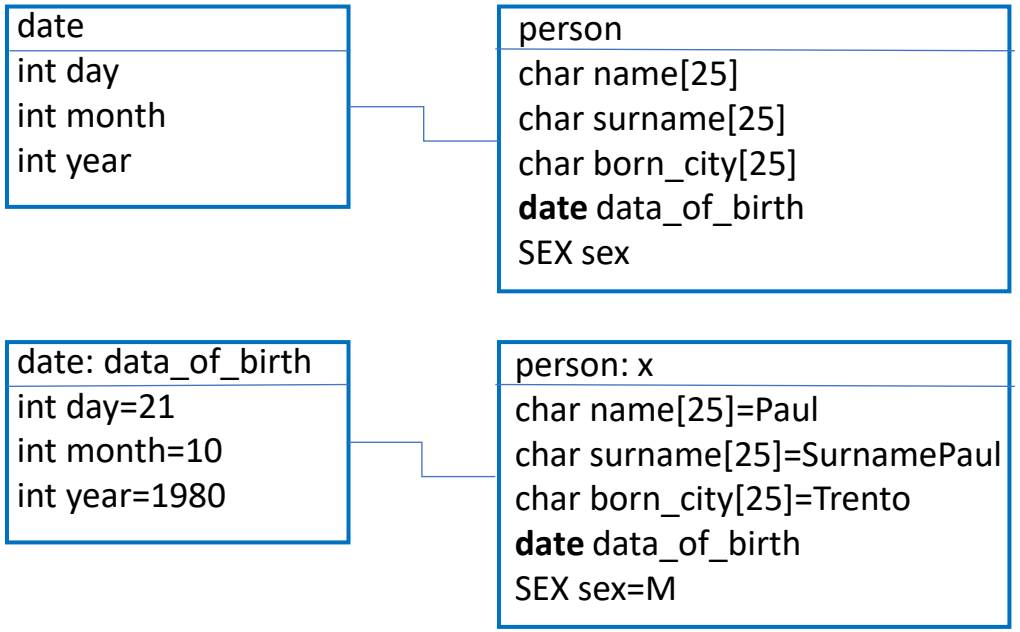
- A variable of type struct is initialized with ordered lists of field values
  - They must match in order and type
  - Any missing values are started at zero of the type

```
struct date {
 int day, month, year;
};


enum SEX { F, M, NotDeclared };
struct person {
 char name[25], surname[25];
 char born_city[25];
 date date_of_birth;
 SEX sex;
};


person x = { “Paul”, “SurnamePaul”, “Trento”, {21, 10, 1980}, M };
person x = { "Paul", "SurnamePaul", "Trento", {}, M };
```

WRONG!



Example:

{ ../STRUCT/testzero.cc, struct_initialization.cc}

Access to the field of a Struct (1)

- If *s* is a struct and *fi* is an identifier for one of its fields, then ***s.fi*** denotes the field *fi* of the struct *s*
  - Syntax: s.fi is an expression with an address
    - Where s is a struct and fi a field of s

  → can be read with >>, assigned, passed by reference, etc.

- If *ps* is a pointer to a structure with *fi* as a field, then it is possible to write *ps->fi* instead of *(*ps).fi*
  - Syntax: *ps->fi* or *(*ps).fi*
    - Where *ps* is a pointer to a structure and *fi* is a field of the structure
  - syntactic sugar but very frequently used

Example: struct

```
1.  using namespace std;
2.  #include <iostream>

3.  struct Person
4.  {
5.    int citizenship;
6.    int age;
7.  };

8.  int main(void) {
9.    Person p;
10.   p.citizenship = 1;
11.   p.age = 27;
12.   cout << "Person citizenship: " << p.citizenship << endl;
13.   cout << "Person age: " << p.age << endl

14.   return 0;
15. }
```

```
1
27
```

Example pointer to a struct

```
1.  using namespace std;
2.  #include <iostream>

3.  struct Length
4.  {
5.    int meters;
6.    float centimeters;
7.  };

8.  int main(){
9.    Length *ptr, l;
10.   ptr = &l;
11.   cout << "Enter meters: ";
12.   cin >> (*ptr).meters;
13.   cout << "Enter centimeters: ";
14.   cin >> (*ptr).centimeters;
15.   cout << "Length = " << (*ptr).meters << " meters " << (*ptr).centimeters << " centimeters";

16.   return 0;
17. }
```

```
5

17
```

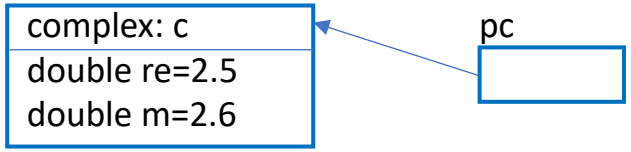
```
5    17
```

Example 1

```
1.  struct complex { double re, m; };

2.  int main(){
3.    complex c;
4.    c.re = 2.5;
5.    c.m = 2.6;
6.    cout << "  c.re: " << c.re << ", c.m: " << c.m << end
7.    complex *pc = &c;
8.    pc->re = 3.5;
9.    pc->m = 3.6;
10.   cout << "  c.re: " << c.re  << ", c.m: " << c.m << endl;
11.   cout << " (*pc).re: " << (*pc).re << ", (*pc).m: " << (*pc).m << endl;
12.   c.m = 2.7;
13.   cout << "  c.re: " << c.re  << ", c.m: " << c.m << endl;
14.   cout << "  pc->re: " << pc->re << ", pc->m:" << pc->m << endl;
15.   return 0;
16. }
```

c.re: 2.5, c.m: 2.6



Example:

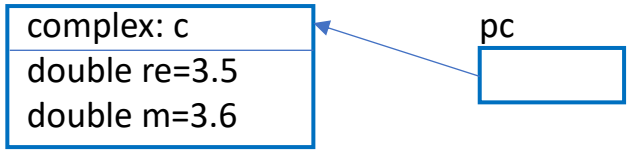
{ ../STRUCT/struct2_bis.cc}

Example 1

```
1.  struct complex { double re, m; };

2.  int main(){
3.    complex c;
4.    c.re = 2.5;
5.    c.m = 2.6;
6.    cout << "  c.re: " << c.re << ", c.m: " << c.m << end
7.    complex *pc = &c;
8.    pc->re = 3.5;
9.    pc->m = 3.6;
10.   cout << "  c.re: " << c.re  << ", c.m: " << c.m << endl;
11.   cout << " (*pc).re: " << (*pc).re << ", (*pc).m: " << (*pc).m << endl;
12.   c.m = 2.7;
13.   cout << "  c.re: " << c.re  << ", c.m: " << c.m << endl;
14.   cout << "  pc->re: " << pc->re << ", pc->m:" << pc->m << endl;
15.   return 0;
16. }
```

c.re: 2.5, c.m: 2.6



Example 1

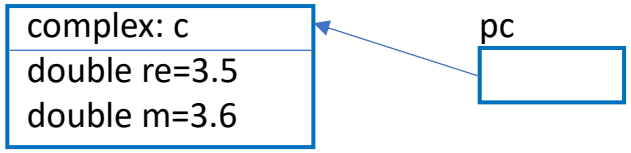
```
1.  struct complex { double re, m; };

2.  int main(){
3.    complex c;
4.    c.re = 2.5;
5.    c.m = 2.6;
6.    cout << "  c.re: " << c.re << ", c.m: " << c.m << end
7.    complex *pc = &c;
8.    pc->re = 3.5;
9.    pc->m = 3.6;
10.   cout << "  c.re: " << c.re  << ", c.m: " << c.m << endl;
11.   cout << " (*pc).re: " << (*pc).re << ", (*pc).m: " << (*pc).m << en
12.   c.m = 2.7;
13.   cout << "  c.re: " << c.re  << ", c.m: " << c.m << endl;
14.   cout << "  pc->re: " << pc->re << ", pc->m:" << pc->m << endl;
15.   return 0;
16. }
```

c.re: 2.5, c.m: 2.6

c.re: 3.5, c.m: 3.6

(*pc).re: 3.5, (*pc).m: 3.6



Example 1

```
1.  struct complex { double re, m; };

2.  int main(){
3.    complex c;
4.    c.re = 2.5;
5.    c.m = 2.6;
6.    cout << "  c.re: " << c.re << ", c.m: " << c.m << end
7.    complex *pc = &c;
8.    pc->re = 3.5;
9.    pc->m = 3.6;
10.   cout << "  c.re: " << c.re  << ", c.m: " << c.m << endl;
11.   cout << " (*pc).re: " << (*pc).re << ", (*pc).m: " << (*pc).m << en
12.   c.m = 2.7;
13.   cout << "  c.re: " << c.re  << ", c.m: " << c.m << endl;
14.   cout << "  pc->re: " << pc->re << ", pc->m:" << pc->m << endl;
15.   return 0;
16. }
```

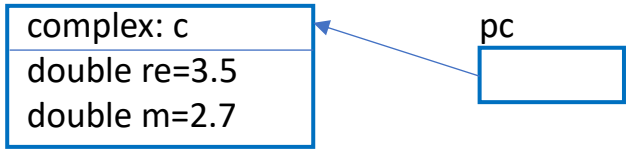
c.re: 2.5, c.m: 2.6

c.re: 3.5, c.m: 3.6

(*pc).re: 3.5, (*pc).m: 3.6

c.re: 3.5, c.m: 2.7

pc->re: 3.5, pc->m: 2.7



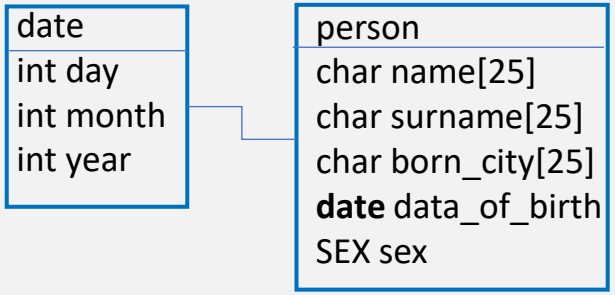
Assignment of a Struct

- Unlike arrays, the assignment between structs is defined
- The assignment of structs is done by value
  - All member values are copied
  - If a field is a “static” array, it is a full copy copied!

→Copying a struct can be computationally **expensive**!

Example

```
person x = { “Paul”, “SurnamePaul”, “Trento”, {21, 10, 1980}, M };
person y=x;          //all fields of x are copied
```



Example:

{ ../STRUCT/struct_assignement.cc, struct_assignement2.cc, struct_assignement3.cc}

Passing a struct to a function

- The structures:
  - They can be passed by value to a function
  - They can be returned from a function via return
- Both operations involve a copy of the member values (including arrays!)

  →Both operations can be computationally **expensive**!

  →Whenever possible, it is preferable to use passage by reference (with const)

Example

```
void print_person1(person p) { … }
void print_person2(const person & p) { … }

person x = { “Paul”, “SurnamePaul”, “Trento”, {21, 10, 1980}, M };
person y = x;

print_person1(x);          // a copy is created
print_person2(y);          // no copy is created
```

person x,y = { “Paul”, “SurnamePaul”, “Trento”, {21, 10, 1980}, M };
→ Only y is assigned!

Example:

{ ../STRUCT/struct3.cc, struct4.cc}

Example 1

```
1.  #include<iostream>
2.  using namespace std;

3.  struct Person
4.  {
5.    int citizenship;
6.    int age;
7.  };

8.  void func(struct Person p1)
9.  {
10.   cout << " Person citizenship: " << p1.citizenship<<endl;
11.   cout << " Person age: " << p1.age;
12. }

13. int main()
14. {
15.   Person p;
16.   p.citizenship = 1;
17.   p.age = 27;
18.   func(p);
19.   return 0;
20. }
```

```
1
27
```

Assignment of static arrays through struct

To handle static arrays, so they can be copied, you can “encapsulate” an array type as a member of a struct

Example

```
struct int_array {
    int a[3];   //static array
};
```

```
int_array sa, sb;   //definition of two structs of type int_array
sa.a[0]=1;
sa.a[1]=2;
sa.a[2]=3;
sb = sa;    // the array is copied (duplication)
```

Example above:

{ ../STRUCT/struct_array.cc}

Example above with initialization:

{ ../STRUCT/struct_array1.cc}

Example 1

```
1.  struct int_array {
2.    int a[3];   //(static) definition of array
3.  };

4.  int main (){

5.   int_array sa, sb;
6.   sa.a[0]=1;
7.   sa.a[1]=2;
8.   sa.a[2]=3;
9.   sb = sa;  //the array sa is copied into sb (all fields, also the array)

10.  cout << "sa = [";
11.  cout << sa.a[0] << " “ << sa.a[1] << " " << sa.a[2] << "]" << endl;
12.  cout << "sb = [";
13.  cout << sb.a[0] << " " << sb.a[1] << " "  << sb.a[2] << "]" << endl;
```

```
14.  sa.a[0]=2;     //changes on sa fields does not impact sb fields
15.  sa.a[1]=3;
16.  sa.a[2]=4;
17.
18.  cout << "sa = [";
19.  cout << sa.a[0] << " " << sa.a[1] << " "  << sa.a[2] << "]" << endl;
20.  cout << "sb = [";
21.  cout << sb.a[0] << " " << sb.a[1] << " "  << sb.a[2] << "]" << endl;

22.  return 0;
23.  }
```

Example 1

```
1.  struct int_array {
2.    int a[3];   //(static) definition of array
3.  };

4.  int main (){

5.   int_array sa, sb;
6.   sa.a[0]=1;
7.   sa.a[1]=2;
8.   sa.a[2]=3;
9.   sb = sa;  //the array sa is copied into sb (all fields, also the array)

10.  cout << "sa = [";
11.  cout << sa.a[0] << " “ << sa.a[1] << " " << sa.a[2] << "]" << endl;
12.  cout << "sb = [";
13.  cout << sb.a[0] << " " << sb.a[1] << " "  << sb.a[2] << "]" << endl;
```

```
sa = [1 2 3]
sb = [1 2 3]
```

```
14.  sa.a[0]=2;     //changes on sa fields does not impact sb fields
15.  sa.a[1]=3;
16.  sa.a[2]=4;
17.
18.  cout << "sa = [";
19.  cout << sa.a[0] << " " << sa.a[1] << " "  << sa.a[2] << "]" << endl;
20.  cout << "sb = [";
21.  cout << sb.a[0] << " " << sb.a[1] << " "  << sb.a[2] << "]" << endl;

22.  return 0;
23.  }
```

Example 1

```
1.  struct int_array {
2.    int a[3];   //(static) definition of array
3.  };

4.  int main (){

5.   int_array sa, sb;
6.   sa.a[0]=1;
7.   sa.a[1]=2;
8.   sa.a[2]=3;
9.   sb = sa;  //the array sa is copied into sb (all fields, also the array)

10.  cout << "sa = [";
11.  cout << sa.a[0] << " “ << sa.a[1] << " " << sa.a[2] << "]" << endl;
12.  cout << "sb = [";
13.  cout << sb.a[0] << " " << sb.a[1] << " "  << sb.a[2] << "]" << endl;
```

```
sa = [1 2 3]
sb = [1 2 3]
```

```
14.  sa.a[0]=2;     //changes on sa fields does not impact sb fields
15.  sa.a[1]=3;
16.  sa.a[2]=4;
17.
18.  cout << "sa = [";
19.  cout << sa.a[0] << " " << sa.a[1] << " "  << sa.a[2] << "]" << endl;
20.  cout << "sb = [";
21.  cout << sb.a[0] << " " << sb.a[1] << " "  << sb.a[2] << "]" << endl;

22.  return 0;
23.  }
```

```
sa = [2 3 4]
sb = [1 2 3]
```

Assignment of dynamic arrays through struct

In the case of dynamic arrays, the pointer is copied

Example **Later in course**

```
struct int_array {
  int *a;    //dynamic array definition
};
int_array sa;
sa.a = new int[3];    //dynamic array instantiation
sb = sa;              // the pointer is copied, sa.a and sb.a are the same array!
```

Example:

{ ../STRUCT/struct_arraypoint.cc, struct_arraypoint2.cc, struct_arraypoint3.cc}

Recursive structures

- The following definition is not valid

```
struct S {
  int value;
  S next;          //circular definition!
};
```

- The following definition is valid

```
struct S {
  int value;
  S *next;
};
```

- Each pointer occupies the same memory space, regardless of its type.
- It is very important for dynamic data structures

Example:

{ ../STRUCT/rec_struct.cc, rec_struct_err.cc}

Example of recursive structure

Example

```
struct S {
  int value;
  S *next; };

int main ()
{
  // definition of two structures of type S
  S str1;
  S str2;

  //initialization of the fields of the first structure
  str1.value = 10;
  str1.next = NULL;

  //initialization of the fields of the second structure
  str2.value = 20;
  str2.next = NULL;

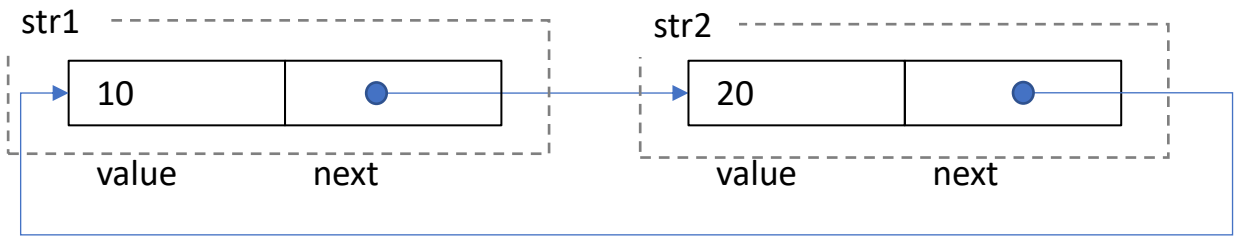
  //creation of a circular link between the two structures
  str1.next = &str2;
  str2.next = &str1;
```

Example (cont.)

```
  //access to the fields of the structures
  cout << "(1) str1 value = " << str1.value << endl;
  cout << "(1) str2 value = " << str2.value << endl;
  cout << "(2) str1 next = " << str1.next << endl;
  cout << "(2) str2 next = " << str2.next << endl;
  //access to the fields of the structures, via the link
  cout << "(3) str2 value (by str1 link to str2) =" << str1.next->value << endl;
  cout << "(3) str1 value (by str2 link to str1) =" << str2.next->value << endl;

  return 0;
}
```

10
20
address of str2
address of str1
20
10



Example:

{ ../STRUCT/rec_struct_advanced.cc}

Mutually recursive structures (1)

- The following definition is not valid

```
struct S1{
  int value;
  S2 *next;
};    //S2 is not defined yet!

struct S2{
  int value;
  S1 *next;
};
```

Example:

{ ../STRUCT/mutrec_struct.cc, mutrec_struct_err.cc}

Mutually recursive structures (2)

- The following definition is not valid

```
struct S1{
  int value;
  S2 *next;
};    //S2 is not defined yet!

struct S2{
  int value;
  S1 *next;
};
```

- The following definition is valid

```
struct S2;      //declaration of S2

struct S1{
  int value;
  S2 *next;
};    //Ok!

struct S2{        //definition of S2
  int value;
  S1 *next;
};
```

Example:

{ ../STRUCT/mutrec_struct.cc, mutrec_struct_err.cc}

Use of an ordered array of structures



- It is common to have to manage organized archives of complex objects (e.g., persons, articles, books, etc.)
  - Basic element of information systems (e.g., archives, inventories, address books, registries)
  - The ordering uses some specific field (key) of the objects (e.g., title of a book)
- Typically used ad-hoc dynamic data structures (e.g., binary search trees)
- Simplified archive example: sorted array of people

Example 1: ordered array of Persons

- We define an array of Persons

  person arr_persons [NmaxPers];

- Ordering the array is done by applying strcpm on the field surname of each person

```
struct person {
  char name[NmaxName];
  char surname[NmaxSurname];
  address address_field;
};
struct group {
  int npers;
  person arr_persons[NmaxPers];
};
```

```
1.  // insert a new person
2.  bool insert_person(const person & p, group & g){
3.    bool res = true;
4.    if (g.npers==NmaxPers)
5.      res = false;
6.    else {
7.      g.arr_persons[g.npers]=p;
8.      g.npers++;
9.      order_for_surname(g);  //procedure to order arr_persons
10.   }
11.   return res;
12. }
```

Example 1: ordered array of Persons

- We define an array of Persons

  person arr_persons [NmaxPers];

- Ordering the array is done by applying strcpm on the field surname of each person

```
struct person {
  char name[NmaxName];
  char surname[NmaxSurname];
  address address_field;
};
struct group {
  int npers;
  person arr_persons[NmaxPers];
};
```

```
1.  // insert a new person
2.  bool insert_person(const person & p, group & g){
3.    bool res = true;
4.    if (g.npers==NmaxPers)
5.      res = false;
6.    else {
7.      g.arr_persons[g.npers]=p;
8.      g.npers++;
9.      order_for_surname(g);  //pr
10.   }
11.   return res;
12. }
```

```
1.  //procedure of bubble sort, by surname
2.  void order_for_surname(group & g){
3.    int i, j;
4.    for (i=g.npers-1; i > 0; i--)
5.      for (j=0; j< i; j++)
6.          if (strcmp(g.arr_persons[j].surname, g.arr_persons[j+1].surname)> 0)
7.              swap(g.arr_persons[j],g.arr_persons[j+1]); // procedure to swap
8.  }
```